RAPPORT DE STAGE

Administration systèmes et DevOps

28 Avril 2025-25 JUIN 2025

AW Services – InnovQube

Présenté et soutenu par

Mme. ONGOLO MBANG Honorée Coeurly

Sous la direction de

M. PIGNARD Arnaud
Président Directeur Général de AW Services

Table of Contents

A PROPOS DE MOI 2
REMERCIEMENT 3
EQUIPE 4
1. Organisme d'accueil 4
1.1. Présentation et Organigramme 4
OUTILS PROFESSIONNELS 6
Administration Réseau & DevOps 6
Système d'exploitation : Linux (Debian 12) 6
Shell / Bash 7
Docker & Docker Compose 7
Redis Cluster 8
HAProxy 8
Keepalived 10
ChartDB 10
OUTILS COLLABORATIFS
Gitlab 11
Outline .....
Openproject .....
Nextcloud 1
MISSIONS 11
MISSION :Test Redis Cluster avec Docker compose 12
Travail effectué et difficultés rencontrées 13
Objectifs des tests : 13
Difficultés rencontrées 15

MISSION3 : Test de Redis Cluster avec Keepalived avec Haproxy 16
Travail éffectué et difficultés rencontrées
MISSION4 : TEST D'INFISICAL 20
Travail éffectué et difficultés rencontrées

CONCLUSION ET PERSPECTIVES 37
WEBOGRAPHIE 38

A PROPOS DE MOI

Je me m'appelle ONGOLO MBANG Honorée Coeurly, étudiante de 17 ans en deuxième année de BTS SIO, option **S**ISR (Solutions d'Infrastructure, Systèmes et Réseaux) à l'IPSSI – Grande École d'Informatique située à Saint-Quentin-en-Yvelines.

Originaire du Gabon, j'ai fais le choix de poursuivre mes études en France afin de développer des compétences solides et concrètes dans le domaine de l'informatique, dans l'optique de m'orienter vers une carrière qui corresponde pleinement à mes aspirations professionnelles.

Dans le cadre de ma formation, j'ai éffectué un **s**tage de deux mois en tant qu'Administratrice Systèmes et DevOps au sein de l'entreprise INNOVQUBE, stage qui m'a permit de mettre en pratique les connaissances acquises en cours et de renforcer mon expertise technique sur le terrain

REMERCIEMENT

Je tiens à exprimer ma profonde gratitude envers toutes les personnes qui ont contribué à la réalisation de mes projets et qui m'ont soutenu tout au long de ce parcours.

Avant tout, je remercie ma famille pour son amour infini et son soutien constant.

Je souhaite également remercier chaleureusement Monsieur Arnaud PIGNARD, PDG d'InnovQube, pour m'avoir offert cette opportunité exceptionnelle de stage. Sa générosité et son accueil chaleureux ont rendu cette expérience enrichissante et mémorable. Je remercie aussi toute l'équipe d'InnovQube pour leur collaboration et leur soutien. Leur expertise et leur disponibilité ont grandement contribué à la réussite à mon adaptation.Je n'oublie pas mes collègues de travail qui ont su faire preuve d'un professionnalisme incontestable.

EQUIPE

1. Organisme d'accueil

1.1. Présentation et Organigramme

InnovQube est un cabinet de conseil spécialisé dans l'accompagnement des entreprises dans leur transformation digitale et l'adoption des technologies cloud. L'objectif est d'aider les clients à tirer pleinement parti du potentiel du cloud Computing pour gagner en agilité, en performance et en compétitivité.

Une équipe d'experts certifiés guide les entreprises à chaque étape de leurs projets cloud, en offrant des services complets tels que :

- La définition de la stratégie cloud et l'élaboration de la roadmap de migration.
- La sélection des solutions et des fournisseurs cloud adaptés aux besoins spécifiques.
- La migration des applications et des données vers le cloud, y compris AWS, Azure, Google Cloud, OVHCloud, et autres.
- L'optimisation des architectures cloud et des coûts grâce aux pratiques de FinOps.
- La sécurisation des environnements cloud et la mise en conformité avec les standards de sécurité.
- La conduite du changement et la formation des équipes pour garantir une adoption réussie des nouvelles technologies.

L’organigramme de l’entreprise repose sur les différentes fonctions exercées au sein de l’organisation.

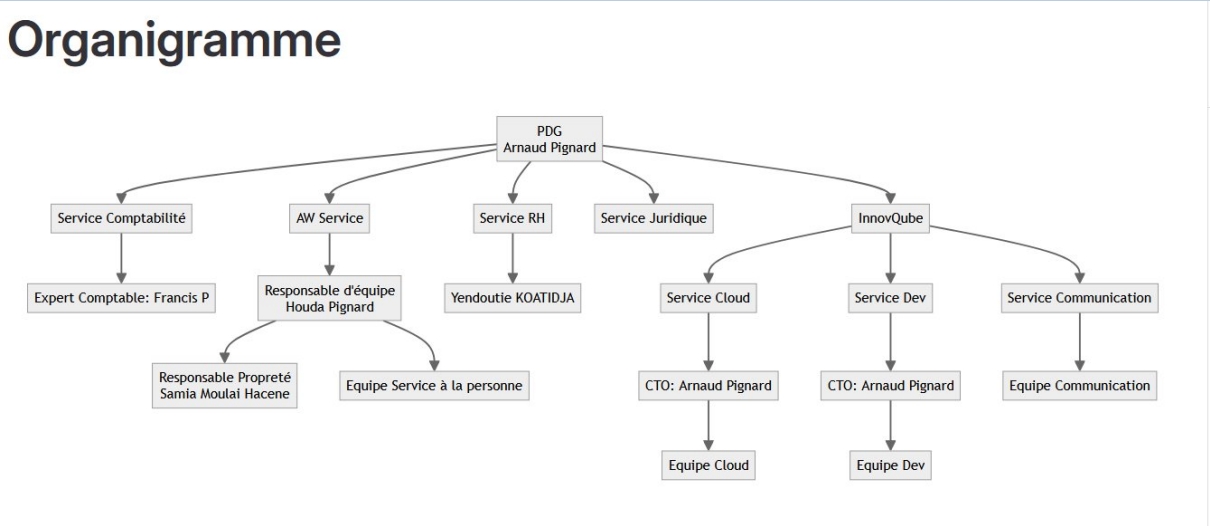

Figure 1: Organigramme de InnovQube

OUTILS PROFESSIONNELS

Administration Réseau & DevOps

Au cours de mon stage chez **Innovqube**, j'ai été amenée à travailler avec une variété d'outils et de technologies liés à l'administration système, la virtualisation, la supervision, le déploiement d'applications conteneurisées et la gestion de la sécurité des secrets. Voici une description détaillée des outils utilisés et leur rôle dans les projets réalisés :

Système d'exploitation : Linux (Debian 12)

Debian a été le système principal sur lequel j'ai effectué mes travaux. J'ai appris à administrer un système Linux via le terminal (Shell Bash), gérer les services, les fichiers de configuration, les utilisateurs, les paquets logiciels, et à dépanner certains problèmes réseau.

```
root@wave-vm:/var/www# apt install php8.3-gd
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libabsl20220623t64 libavif16 libdav1d7 libgav1-1 libgd3 libimagequant0 libraqm0 librav1e0 libsvtav1enc1d1 libxpm4 libyuv0
Suggested packages:
  libgd-tools
The following NEW packages will be installed:
  libabsl20220623t64 libavif16 libdav1d7 libgav1-1 libgd3 libimagequant0 libraqm0 librav1e0 libsvtav1enc1d1 libxpm4 libyuv0 php8.3-gd
0 upgraded, 12 newly installed, 0 to remove and 1 not upgraded.
Need to get 5350 kB of archives.
After this operation, 16.9 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://archive.ubuntu.com/ubuntu noble/main amd64 libabsl20220623t64 amd64 20220623.1-3.1ubuntu3 [423 kB]
Get:2 https://ppa.launchpadcontent.net/ondrej/php/ubuntu noble/main amd64 libgd3 amd64 2.3.3-12+ubuntu24.04.1+deb.sury.org+1 [130 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble/universe amd64 libdav1d7 amd64 1.4.1-1build1 [604 kB]
Get:4 https://ppa.launchpadcontent.net/ondrej/php/ubuntu noble/main amd64 php8.3-gd amd64 8.3.19-1+ubuntu24.04.1+deb.sury.org+1 [31.5 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble/universe amd64 libgav1-1 amd64 0.18.0-1build3 [357 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/universe amd64 librav1e0 amd64 0.7.1-2 [1022 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble/universe amd64 libsvtav1enc1d1 amd64 1.7.0+dfsg-2build1 [2425 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble/universe amd64 libyuv0 amd64 0.0~git202401110.af6ac82-1 [178 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble/universe amd64 libavif16 amd64 1.0.4-1ubuntu3 [91.2 kB]
Get:10 http://archive.ubuntu.com/ubuntu noble/main amd64 libimagequant0 amd64 2.18.0-1build1 [36.3 kB]
Get:11 http://archive.ubuntu.com/ubuntu noble/main amd64 libraqm0 amd64 0.10.1-1build1 [15.0 kB]
Get:12 http://archive.ubuntu.com/ubuntu noble/main amd64 libxpm4 amd64 1:3.5.17-1build2 [36.5 kB]
Fetched 5350 kB in 1s (4155 kB/s)
Selecting previously unselected package libabsl20220623t64:amd64.
(Reading database ... 39583 files and directories currently installed.)
Preparing to unpack .../00-libabsl20220623t64_20220623.1-3.1ubuntu3_amd64.deb ...
Unpacking libabsl20220623t64:amd64 (20220623.1-3.1ubuntu3) ...
Selecting previously unselected package libdav1d7:amd64.
Preparing to unpack .../01-libdav1d7_1.4.1-1build1_amd64.deb ...
Unpacking libdav1d7:amd64 (1.4.1-1build1) ...

Progress: [  65%] [#########.....................................................]
```

Shell / Bash

Le Shell a été au cœur de toutes les opérations quotidiennes. J'ai utilisé **Bash** pour :

- naviguer dans l'arborescence des fichiers,
- modifier les permissions,
- installer des paquets,
- exécuter des scripts d'automatisation,
- interagir avec des outils comme Docker, Infisical ou Git.

```
root@wave-vm:~# mkdir /var/www
root@wave-vm:~# cd /var/www/
root@wave-vm:/var/www# apt update && apt upgrade -y
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 https://download.docker.com/linux/ubuntu noble InRelease [48.8 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-security InRelease [126 kB]
0% [Working]
```

Docker & Docker Compose

Docker m’a permis de déployer des applications de manière isolée et portable. J’ai utilisé

- **Docker Compose** pour orchestrer plusieurs conteneurs (notamment pour Redis Cluster),
- gestion des volumes, des réseaux, des images et des variables d’environnement.

Redis Cluster

J’ai mis en place un **cluster Redis** pour assurer la **réplication** et la **haute disponibilité** des données.
Ce projet m’a permis de comprendre les notions de sharding, master/slave et de communication inter-nœuds dans un environnement distribué.

HAProxy

Outil essentiel pour le **load balancing**, j’ai configuré HAProxy pour distribuer la charge entre plusieurs nœuds Redis. Il permet d'assurer la continuité de service même en cas de défaillance d’un nœud.

Démarrage du cluster avec docker-compose up -d et vérification du cluster

```
root@honor13:/home/redis/redis-server# docker-compose ps
        Name                      Command              State              Ports
-------------------------------------------------------------------------------------
redis-cluster-setup   sh -c sleep 10 && redis-    Exit 0
                      cl ...
redis-replica-1       docker-entrypoint.sh        Up       0.0.0.0:6382-
                      redis ...                            >6379/tcp,:::6382-
                                                           >6379/tcp
redis-replica-2       docker-entrypoint.sh        Up       0.0.0.0:6383-
                      redis ...                            >6379/tcp,:::6383-
                                                           >6379/tcp
redis-replica-3       docker-entrypoint.sh        Up       0.0.0.0:6384-
                      redis ...                            >6379/tcp,:::6384-
                                                           >6379/tcp
redis1                docker-entrypoint.sh        Up       0.0.0.0:6379-
                      redis ...                            >6379/tcp,:::6379-
                                                           >6379/tcp
redis2                docker-entrypoint.sh        Up       0.0.0.0:6380-
                      redis ...                            >6379/tcp,:::6380-
                                                           >6379/tcp
redis3                docker-entrypoint.sh        Up       0.0.0.0:6381-
                      redis ...                            >6379/tcp,:::6381-
                                                           >6379/tcp
```

Plan de test unitaire

```
root@honor13:/home/redis/redis-server# docker exec -it redis1 redis-cli cluster node
s
eafd0eecd91c2554c5fdd716c4f4e93dd3bb1bd8 172.18.0.2:6379@16379 myself,master - 0 0 1
 connected 0-5460
dd1f849cc3ee2faf39cb6d4a751ac4d7a4dc66be 172.18.0.3:6379@16379 master - 0 1747377540
838 2 connected 5461-10922
822c775e4700e341e8a4aef0627852890fc2c5cc 172.18.0.4:6379@16379 master - 0 1747377541
844 3 connected 10923-16383
74389f8bf217adad08617ca2d278f5bc5e24818e 172.18.0.7:6379@16379 slave dd1f849cc3ee2fa
f39cb6d4a751ac4d7a4dc66be 0 1747377540533 2 connected
181aac9251974c32a73877457b6bfd2484704a26 :0@0 slave,noaddr 822c775e4700e341e8a4aef06
27852890fc2c5cc 1747375900027 1747375899975 3 disconnected
a06bdf142d573f2c82cd24f2aed630d7b138b540 172.18.0.6:6379@16379 slave eafd0eecd91c255
4c5fdd716c4f4e93dd3bb1bd8 0 1747377541000 1 connected
root@honor13:/home/redis/redis-server# docker ps -a | grep 181aac9251974c32a7
```

Keepalived

Keepalived m'a permis d'ajouter une couche de **haute disponibilité (HA)** au niveau IP, grâce à la mise en place du protocole VRRP. En cas de panne de la VM principale, l'adresse IP flottante est reprise par une autre VM.

Vérification des logs HAProxy / Keepalived ✅ et test de bascule

Test de bascule haute disponibilité (HA) ✅.

Le logs haproxy deviennent

```
root@honor13:/home/redis/redis-server# docker stop haproxy-master
```

```
May 16 09:07:21 honor13 Keepalived_vrrp[4096]: (VI_1) received
!
May 16 09:07:22 honor13 Keepalived_vrrp[4096]: (VI_1) Entering
May 16 09:07:23 honor13 Keepalived_vrrp[4096]: (VI_1) received
```

Les logs haproxy Master deviennent

```
root@honor13:/home/redis/redis-server# docker logs -f haproxy-master
2025-05-19T16:09:45.525262+00:00 honor13 Keepalived[18]: Starting Keepalived v2.3.1 (05/24,2024)
2025-05-19T16:09:45.525558+00:00 honor13 Keepalived[18]: Running on Linux 6.8.12-10-pve #1 SMP PREEMPT_DYNAMIC PMX 6.8.12-10 (2025-04-18T07:39Z) (built for Linux 6.6.0)
2025-05-19T16:09:45.525605+00:00 honor13 Keepalived[18]: Command line: 'keepalived' '-i' 'main'
```

ChartDB

J'ai déployé **ChartDB**, une base de données utilisée pour la visualisation de données. Ce déploiement m'a familiarisée avec la configuration des applications basées sur des API, ainsi que la gestion des ports, accès et dépendances dans un environnement conteneurisé.

OUTILS COLLABORATIFS

Gitlab

Plateforme DevOps intégrée comprenant la gestio de référentiels Git, des pipelines CI/CD le suivi des problèmes et la révision de code Optimisant ainsi le flux de travail et assurant la qualité des livrables.

plateforme de cloud computing open source utilisée pour le stockage et le partage sécurisé de fichiers, offrant des fonctionnalités de synchronisation et d'édition en
ligne

Outline Outil de création de wikis facilitant la rédaction, l'organisation et la recherche de
documents, centralisant les connaissances et les procédures de l'entreprise

MISSIONS

MISSION1 :Deploiement de Wave

Wave est une plateforme qui permet de créer rapidement un Software as a Service (SaaS) en offrant une multitude de fonctionnalités prêtes à l'emploi, ce qui simplifie grandement le développement d'une application SaaS. Voici les principales fonctionnalités que Wave propose :

1. Authentification intégrée : Vous n'avez pas à créer un système d'authentification utilisateur à partir de zéro. Wave fournit une solution d'authentification prête à l'emploi avec la gestion des utilisateurs et des rôles.
2. Abonnements et facturation : Il prend en charge les abonnements récurrents et permet de gérer les factures automatiquement. Cela est essentiel pour les services SaaS qui nécessitent des paiements réguliers de la part des utilisateurs.
3. Annonces et notifications : Vous pouvez envoyer des annonces aux utilisateurs et gérer les notifications de manière centralisée.
4. Profils utilisateur : Chaque utilisateur peut avoir un profil personnalisé avec des informations détaillées.
5. API : Wave facilite la création d'API pour permettre l'intégration de votre service avec d'autres applications ou services.
6. Fonctionnalités supplémentaires : Wave inclut également des outils pour la gestion des équipes, la gestion des permissions, la génération de rapports, etc. Avantages pour les développeurs : • Gain de temps : Les fonctionnalités cruciales pour un SaaS sont déjà intégrées. • Personnalisable : Bien que Wave fournisse des solutions par défaut, vous pouvez personnaliser le tout selon les besoins de votre projet.

Travail et difficultés rencontrées :

Ma mission était de déployer cette solution afin de pouvoir m'inprégner des outils utilisés au sein de l'entreprise.

Lors du déploiement de Wave, j'ai pu mettre en pratique les compétences acquises en cours. Toutefois, j'ai rencontré certaines difficultés lors de la migration de la base de données, notamment en raison d'un problème lié à une extension. Ce dernier a cependant été rapidement résolu.

MISSION2 :Test Redis Cluster avec Docker compose

Redis est une base de données NoSQL clé-valeur largement utilisée pour des cas d'usage tels que la mise en cache, la gestion de sessions et les files de messages. Dans le cadre d'une architecture micro-services ou web hautement performants, Redis doit être hautement disponible, résilient aux pannes, et capable de monter en charge horizontalement. Ce document décrit le déploiement d'une solution Redis résiliente et scalable en utilisant des outils comme HAProxy, Docker, et une architecture de type Redis Cluster pour garantir la haute disponibilité et la répartition de la charge.

Voici une version réécrite, harmonisée et plus fluide de votre texte, adaptée pour un rapport technique ou de stage :

1MISSION3 :DEPLOIEMENT CHARTDB

1

Travail effectué et difficultés rencontrées

Dans le cadre de ma mission, j'ai été chargé de réaliser des plans de tests unitaires pour le déploiement d'un cluster Redis à l'aide de Docker Compose, sur une architecture composée d'une machine **MASTER** et d'une **SLAVE**.

Objectifs des tests :

1. **Vérification du bon fonctionnement des services**
    - Lancer l'ensemble des conteneurs Redis.
    - Vérifier que tous les services démarrent correctement.

```
root@honor13:/home/redis/redis-server# docker-compose ps
        Name                      Command              State              Ports
-----------------------------------------------------------------------------------------
redis-cluster-setup   sh -c sleep 10 && redis-    Exit 0
                      cl ...
redis-replica-1       docker-entrypoint.sh        Up       0.0.0.0:6382-
                      redis ...                            >6379/tcp,:::6382-
                                                           >6379/tcp
redis-replica-2       docker-entrypoint.sh        Up       0.0.0.0:6383-
                      redis ...                            >6379/tcp,:::6383-
                                                           >6379/tcp
redis-replica-3       docker-entrypoint.sh        Up       0.0.0.0:6384-
                      redis ...                            >6379/tcp,:::6384-
                                                           >6379/tcp
redis1                docker-entrypoint.sh        Up       0.0.0.0:6379-
                      redis ...                            >6379/tcp,:::6379-
                                                           >6379/tcp
redis2                docker-entrypoint.sh        Up       0.0.0.0:6380-
                      redis ...                            >6379/tcp,:::6380-
                                                           >6379/tcp
redis3                docker-entrypoint.sh        Up       0.0.0.0:6381-
                      redis ...                            >6379/tcp,:::6381-
                                                           >6379/tcp
```

2. **Formation du cluster Redis (sur VM1)**
    - S'assurer que les 6 nœuds Redis (3 masters et 3 réplicas) sont correctement créés.
    - Valider l'allocation correcte des slots de hash et la configuration des relations maître/réplique.

```
root@honor13:/home/redis/redis-server# docker exec -it redis1 redis-cli cluster node
s
eafd0eecd91c2554c5fdd716c4f4e93dd3bb1bd8 172.18.0.2:6379@16379 myself,master - 0 0 1
 connected 0-5460
dd1f849cc3ee2faf39cb6d4a751ac4d7a4dc66be 172.18.0.3:6379@16379 master - 0 1747377540
838 2 connected 5461-10922
822c775e4700e341e8a4aef0627852890fc2c5cc 172.18.0.4:6379@16379 master - 0 1747377541
844 3 connected 10923-16383
74389f8bf217adad08617ca2d278f5bc5e24818e 172.18.0.7:6379@16379 slave dd1f849cc3ee2fa
f39cb6d4a751ac4d7a4dc66be 0 1747377540533 2 connected
181aac9251974c32a73877457b6bfd2484704a26 :0@0 slave,noaddr 822c775e4700e341e8a4aef06
27852890fc2c5cc 1747375900027 1747375899975 3 disconnected
a06bdf142d573f2c82cd24f2aed630d7b138b540 172.18.0.6:6379@16379 slave eafd0eecd91c255
4c5fdd716c4f4e93dd3bb1bd8 0 1747377541000 1 connected
root@honor13:/home/redis/redis-server# docker ps -a | grep 181aac9251974c32a7
```

3. **État du cluster**
    - Vérifier la stabilité, la complétude et la cohérence globale du cluster.

4. **Test de la réplication**
    - Confirmer que chaque master dispose d'au moins une réplique connectée.

- Vérifier que la valeur `connected_slaves` est supérieure à 0 et que l'état de chaque esclave est `online`.

5. **Test de connectivité**
   - Valider que le cluster est accessible via le proxy.
6. **Vérification du routage automatique**
   - S'assurer que les requêtes sont correctement routées à travers les différents nœuds du cluster.

```
root@honor13:/home/redis/redis-server# docker exec redis1 redis-cli PING
PONG
root@honor13:/home/redis/redis-server# docker exec -it redis1 redis-cli -c set user1
 "Toto"
docker exec -it redis1 redis-cli -c get user1
OK
"Toto"
root@honor13:/home/redis/redis-server#
```

7. **Simulation de panne (Failover)**
   - Tester le mécanisme d'auto-bascule en cas de défaillance d'un nœud maître, et s'assurer que l'un des esclaves prend correctement le relais.

Difficultés rencontrées

Durant cette phase de test, je n'ai rencontré aucunnes difficultés tout était opérationnel.

MISSION3 : Test de Redis Cluster avec Keepalived avec Haproxy

Architecture

Description de l'architecture

L'architecture repose sur des conteneurs Docker orchestrés via Docker Compose, avec pour objectif de garantir la **haute disponibilité**, la **résilience aux pannes**, et la **répartition de la charge**.

Composants principaux

1. **redis-master** : Instance Redis principale, responsable des écritures.
2. **redis-slave-N** : Instances Redis esclaves, dédiées aux lectures et répliquées depuis le master.
3. **HAProxy** : Load balancer TCP, répartissant les requêtes entre le master Redis et ses slaves.
4. **Keepalived** : Garantit la haute disponibilité en assurant le basculement de la **VIP (Virtual IP)** entre les nœuds principaux et secondaires en cas de panne.

Vision statique

Vision dynamique

- Les instances Redis sont configurées en cluster, avec des répliques synchronisées entre le **redis-master** et ses **redis-slave-N**.
- **HAProxy** assure le load balancing, dirigeant le trafic en fonction du rôle des nœuds Redis (master ou slave).

MISSION4 : TEST D'INFISICAL

1Introduction

Infisical est une plateforme de gestion de secrets open source que les développeurs utilisent pour centraliser la configuration de leurs applications et leurs secrets, comme les clés API et les informations d'identification de la base de données, ainsi que pour gérer leur PKI interne. De plus, les développeurs utilisent Infisical pour empêcher les fuites de secrets vers Git et partager en toute sécurité les secrets entre les ingénieurs.

Comment fonctionne Infisical ?

Pour rendre la gestion des secrets simple et sécurisée, Infisical suit une certaine structure pour permettre les flux de travail de gestion des secrets tels que définis ci-dessous.

Les identités dans Infisical sont des utilisateurs ou des machines auxquels est attribué un certain ensemble de rôles et d'autorisations. Ces identités sont capables de gérer les secrets de divers **clients** dans l'ensemble de l'infrastructure. Pour ce faire, les identités doivent se vérifier via l'une des **méthodes d'authentification** disponibles .

Par conséquent, les 3 principaux concepts qu'il est important de comprendre sont :

- **Identités** : utilisateurs ou machines disposant d'un ensemble d'autorisations qui leur sont attribuées.
- **Clients** : outils développés par Infisical pour la gestion des secrets dans divers composants d'infrastructure (par exemple, **Kubernetes Operator** , **Infisical Agent** , **CLI** , **SDK** , **API** , **Web Dashboard**).
- **Méthodes d'authentification** : moyens permettant aux identités de s'authentifier au sein de différents clients (par exemple, SAML SSO pour Web Dashboard, Universal Auth pour Infisical Agent, AWS Auth, etc.).

Travail éffectué et difficultés rencontrées

Déploiement de la plateforme Infisical

Dans le cadre de mon stage, j'ai pu procéder avec succès au **déploiement de la plateforme Infisical** en environnement Docker. L'application a été configurée pour utiliser **GitLab**

comme fournisseur d'authentification OAuth2, permettant ainsi une redirection correcte des utilisateurs vers GitLab lors de la connexion.

Cependant, j'ai rencontré un **problème au niveau de l'utilisation de la CLI (interface en ligne de commande) d'Infisical**. Bien que le déploiement de l'interface web et l'authentification fonctionnaient comme prévu, certaines commandes CLI ne répondaient pas correctement, vraisemblablement en raison de problèmes de configuration ou d'authentification entre la CLI et le backend.

Une analyse plus approfondie serait nécessaire pour :

- Vérifier les variables d'environnement requises par la CLI,
- Confirmer la compatibilité de la version CLI avec l'API déployée,
- Et s'assurer de la gestion correcte des jetons d'accès dans un environnement OAuth2 via GitLab

Laravel

Laravel est un Framework d'application web PHP open-source, destiné pour le développement d'applications. Ses principales fonctionnalités sont :

- Composer en tant que gestionnaire de dépendance pour ajouter des packages de structure différentes et spécifiques disponible à partir du référentiel Packagist permettant ainsi le support de la majorité des technologies actuelle.
- Eloquent ORM (mapping objet-relationnel) est une implémentation de PHP avancé du motif d'enregistrement actif, en fournissant en même temps des méthodes internes pour appliquer des contraintes sur les relations entre les objets de base de données.

- Laravel dispose d'un moteur de Template "Blade" qui combine un ou plusieurs modèles avec un modèle de données pour produire des nouvelles vues, en traduisant les modèles dans le code PHP en cache pour améliorer les performances.
- Migrations : fournissent un système de contrôle de version pour les schémas de base

de données, ce qui permet d'associer des changements dans le code source de l'application et les modifications nécessaires à la mise en page de base de données. En conséquence, le déploiement et la mise à jour des applications est considérablement simplifié.

Nous trouvons dans Laravel :

- Un système de routage perfectionné (RESTFUL et ressources),
- Un créateur de requêtes SQL et un ORM performants,
- Un moteur de Template efficace,
- Un système d'authentification pour les connexions,
- Un système de validation,
- Un système de pagination,
- Un système de migration pour les bases de données,
- Un système d'envoi d'emails,
- Un système de cache,
- Une gestion des sessions...

Le pattern implémenté par Laravel est le pattern MVC (Modèle-Vue-Contrôleur). En plus, Laravel est fondamentalement orienté objet. La POO est un paradigme qui s'éloigne radicalement de la programmation procédurale. Avec la POO tout le code est placé dans des classes qui découlent d'interfaces qui établissent des contrats de fonctionnement. Avec la POO on manipule des objets.

En conclusion, Laravel est un Framework très complet et très puissant. Il a la particularité d'être plus permissive et plus souple que son confrère Symfony. Le fait qu'il soit très facile d'accès avec sa documentation complète et sa communauté active rend Laravel très attractif lors du choix d'un Framework.

Wave

Wave est un framework mettant en place un système de gestion des utilisateurs, des rôles et des autorisations. Il fournit une solution prête à l'emploi pour gérer l'authentification, l'inscription des utilisateurs, la réinitialisation des mots de passe, les rôles et les permissions dans une application Laravel.

L'utilisation de Wave nous a permis d'intégrer rapidement des fonctionnalités d'authentification et de gestion des utilisateurs et cela nous a permis de gagner du temps et d'assurer une implémentation sécurisée et robuste des fonctionnalités d'authentification.

Avec la mise à jour de Wave en version 3, Voyager qui est un système d'administration a été remplacé par Filament.

Filament

Filament est une bibliothèque de composants d'interface utilisateur, offrant une solution complète pour gérer les aspects administratifs d'une application Laravel. Filament fournit une interface utilisateur riche et conviviale pour gérer les contenus, les utilisateurs, les paramètres de l'application. Il propose des fonctionnalités telles que la gestion des tableaux de bord, la mise en place d'interfaces personnalisées basées sur CRUD comme les formulaires, et bien plus encore. En parallèle de Filament, nous avons aussi utilisé Flowbite.

Flowbite

Flowbite est une bibliothèque de composants d'interface utilisateur (UI) open-source conçue pour simplifier le développement Web et fournir une base solide pour la création d'interfaces utilisateurs modernes et réactives. Cette bibliothèque offre une gamme de composants prêts à l'emploi, tels que des boutons, des formulaires, des barres de navigation, des cartes et bien d'autres. Flowbite est basé sur HTML, CSS (notamment Tailwind CSS) et Ja-

vaScript. Il est donc facile d'intégrer des éléments dans divers projets Web.

F

CONCLUSION ET PERSPECTIVES

Ce stage chez INNOVQUBE m'a permis de renforcer mes compétences techniques en administration système, en supervision réseau et dans les outils DevOps . J'ai pu travailler sur des projets concrets tels que le déploiement de clusters Redis, et la gestion des secrets avec Infisical, ce qui m'a permis de gagner en rigueur, en autonomie et en méthode. Bien que j'ai parfois rencontrés certaines difficultés j'ai pu apprendre de mes erreurs et remonter la pente.

Cette expérience a confirmé mon envie d'évoluer dans le domaine de l'infrastructure et de la sécurité informatique, en particulier à travers les technologies open source. Elle m'a également permis de mieux comprendre le fonctionnement d'une équipe technique et l'importance

de la documentation et de la collaboration.

Je tiens à remercier toute l'équipe d'INNOVQUBE pour leur accueil et leur disponibilité.

Je poursuis désormais ma formation en BTS SIO – option SISR, avec la volonté de trouver une **alternance de 12 mois à partir de septembre 2025**, afin de continuer à apprendre sur le terrain et d'approfondir mes compétences en administration système, réseaux et DevOps

WEBOGRAPHIE

- Campus France :

https://ressources.campusfrance.org/pratique/etablissements/fr/univ_rochelle_fr.pdf

- Documentation Laravel :

https://laravel.com/docs/10.x

- Documentation wave :

https://wave.devdojo.com/docs

- Filament :

https://derniercri.io/radar-items/filament-php

- PHP && MySQL :

https://www.pearson.fr/resources/titles/27440100333930/extras/2554_intro.pdf

https://www.php.net/manual/fr/intro-whatis.php

- Flowbite :

https://flowbite.com/

- Documentation HTML

https://developer.mozilla.org/fr/docs/Web/HTML

- Framework web :

https://www.lemon-interactive.fr/actualites/developpement/tout-comprendre-framework-web-avantages-inconvenients/

https://neverendingsecurity.wordpress.com/2015/04/22/15-best-free-php-frameworks-of-2015/